# La caja negra · Especificación del servicio

Frontier Labs · Reto 2 · Especificación reimplementable · 82 comportamientos · 394 observaciones · **evidencia verificada**

> Permite reimplementar el servicio **sin tocar el original**. Cada comportamiento cita **IDs exactos** del log de experimentos; `reto2-investigacion/scripts/validar_spec.py` comprueba que la evidencia existe y corresponde. Importa preservar **respuesta + estado posterior**, incluidas las anomalías (typo `recieved`, espacio final de `E301`).

**Especificación de servicio reimplementable.** Cada comportamiento tiene un ID, su descripción, la evidencia (**IDs exactos** de `experimentos.jsonl`) y su confianza. Permite a otro ingeniero reimplementar el servicio **sin tocar el original**.

- **confirmado**: probado y se repite en corridas distintas.
- **probable**: visto una vez o inferido de la evidencia.
- **supuesto**: no probado; es el default del reemplazo.

La evidencia se verifica automáticamente con `reto2-investigacion/scripts/validar_spec.py`: comprueba que cada ID citado existe y que la respuesta observada corresponde al comportamiento.

Secuencias del log: `integridad` (1–16), `aridad` (17–46), `alta` (47–57), `consulta` (58–64), `deposito` (65–86), `transfer` (87–100), `anula` (101–116), `estado` (117–121), `bordes` (122–163), `bloqueo` (164–239), `bloqueo3` (240–278), `max` (279–290), `sugerencia` (291–308), `bloqueo4` (309–336), `sugerencia2` (337–347), `gaps` (348–366), `consulta-v2` (370–373), `transfer-v2` (374–377), `halt` (378–379, 387–388), `bloqueo-v2` (380–383), `ascii` (385–386), `transfer-nombres` (391–394).

## 1 · Entrada

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| I-01 | `GET /health` con token → `200 {"ok":true}`. | 349 | confirmado |
| I-02 | `POST /msg` recibe `{"m":"..."}`. | 1–347, 350–394 | confirmado |
| I-03 | `POST /__reset` → `200 {"ok":true}` y limpia el estado (cuentas, operaciones y contadores; el siguiente `ALTA` vuelve a `AC-0001`). | 117–121, 348, 360 | confirmado |
| I-04 | Cualquier otra ruta/método no definidos → `404 {"error":"E404","mensaje":"ruta desconocida"}` (`GET /msg`, `POST /health`, `GET /__reset`). | 352–354 | confirmado |
| I-05 | Todas las rutas exigen `Authorization: Bearer <token>`; ausente/incorrecto → `401 {"error":"E001","mensaje":"no autorizado"}`. El esquema `Bearer` es sensible a mayúsculas. | 350, 351 | confirmado |
| I-06 | La cabecera `Content-Type` se ignora: se parsea JSON igual. | 361 | confirmado |
| I-07 | Cuerpo > 8191 bytes → `413 {"error":"E101","mensaje":"cuerpo demasiado grande"}`. | 366 | confirmado |
| I-08 | JSON inválido o cuerpo vacío → `400 {"error":"E100","mensaje":"json inválido"}`. | 355, 356 | confirmado |
| I-09 | JSON válido sin `m` string (ausente, número, `null`) → `400 {"error":"E102","mensaje":"falta m"}`. Claves extra se ignoran. | 357–359 | confirmado |
| I-10 | La respuesta incluye cabecera `x-req` (contador global monótono) y `__reset` **no** lo reinicia. | 361–364 | confirmado |
| I-11 | El límite de infraestructura responde `429 {"infra":"rate-limit"}` con `X-Frontier-Infra`; **no** es comportamiento del servicio. | 365 | confirmado |

## 2 · Integridad (dígito de control)

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| C-01 | El último segmento `;`-separado es el **control**. | 1–16 | confirmado |
| C-02 | Sin separador `;` → `400 {"error":"E103","mensaje":"falta control"}`. | 1 | confirmado |
| C-03 | Algún segmento vacío → `400 {"error":"E104","mensaje":"segmento vacío"}` (tiene prioridad sobre el control). | 2, 10, 11, 53, 147 | confirmado |
| C-04 | Si el control no cumple `^\d{1,2}$` (3+ dígitos, signo, espacio, letras) → `400 {"error":"E201","mensaje":"control mal formado"}`. | 7, 8, 9, 144 | confirmado |
| C-05 | Control con formato válido pero ≠ canónico → `400 {"error":"E202","mensaje":"control inválido"}` (`03` sí, `3` también; `003` no). | 6, 145, 150, 151 | confirmado |
| C-06 | El control es `ISO 7064 MOD 97-10` (verificador tipo IBAN) sobre el mensaje **sin su último campo**: cada dígito vale su valor; cada letra `A=1…Z=26` (insensible a mayúsculas); `;`, `-`, espacios y no-ASCII no aportan. `control = 98 - (100·(N mod 97)) mod 97`, sin ceros a la izquierda. | `data/pares_control.jsonl` (34/34), 3, 4, 5, 12 | confirmado |
| C-07 | `PING` **ignora** el valor del control (`PING;0064`, `PING;abc` → `200`), pero sí exige que exista y no esté vacío. | 3, 4 | confirmado |

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| C-08 | Los comandos desconocidos **sí** validan el control antes de decidir que el comando no existe ( `SALDOS;x;abc` → `E201` , no `E301` ). | 144–147, 150, 291–308 | confirmado |

Implementación de referencia:

```
def digitos(cuerpo):
    out = []
    for c in cuerpo:
        if c.isdigit() and c.isascii(): out.append(c)
        elif c.isascii() and c.isalpha(): out.append(str(ord(c.upper()) - 64))
    return "".join(out)

def control(cuerpo):
    r = 0
    for d in digitos(cuerpo): r = (r * 10 + int(d)) % 97
    return 98 - (r * 100) % 97
```

## 3 · Comandos

Orden general de evaluación: cuando el servicio está apagado ( `HALT;1` ), `E599` tiene **precedencia absoluta** (antes que todo). Después: `E001` → `E100` → `E102` → `E103` → `E104` → control ( `E201` / `E202` , salvo `PING` ) → `E301` (comando desconocido) → `E302` (aridad) → reglas de negocio ( `E303` , `E304` , `E401` , `E510` , `E506` , `E507` , `E402` ).

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| K-00 | Comandos válidos (insensibles a mayúsculas): `PING` , `ALTA` , `CONSULTA` , `DEPOSITO` , `TRANSFER` , `ANULA` y, en v2, `HALT` y `NOTAS` . | 3, 5, 12, 66, 90, 104, 378, 379 | confirmado |
| K-00B | El match de comando es insensible a mayúsculas pero **solo ASCII**: `ping` / `PING` funcionan; `ping` (i sin punto, U+0131) **no** (→ `E301` con `sugerencia:"PING"` o `E201` si el control es mal formado). No se recortan espacios ( `PING` , `PING` → `E301` ). | 385, 386 | confirmado |
| K-01 | Aridad exacta (segmentos incluyendo control): `PING` 2, `ALTA` 3, `CONSULTA` 3, `DEPOSITO` 4, `TRANSFER` 5, `ANULA` 3, `HALT` 3, `NOTAS` 2. Si no coincide → `400 E302` . | 17–46, 378, 379 | confirmado |

### PING

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| K-P1 | `PING;<cualquiera>` → `200 {"pong":true}` . | 3, 4 | confirmado |

### ALTA

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| K-A1 | `ALTA;<nombre>;<control>` → `201 {"cuentaId":"AC-000N"}` . El id es secuencial con 4 dígitos ( `AC-%04d` ). | 5, 47 | confirmado |
| K-A2 | El nombre puede ser cualquier segmento no vacío (espacios, dígitos, acentos, guiones, largos). Duplicados crean cuentas nuevas. | 48–52 | confirmado |
| K-A3 | El contador de cuentas lo reinicia `__reset` ; `PING` no lo consume. | 118, 120 | confirmado |

### CONSULTA

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| K-C1 | `CONSULTA;<id>;<control>` existente → `200 {"cuenta_id":"<id>","saldo":"S.DD"}` (v2: saldo como cadena decimal de 2 dígitos; unidades = centavos). | 12, 60, 370, 372 | confirmado |
| K-C2 | Cuenta inexistente → `404 {"error":"E401","mensaje":"cuenta inexistente"}` . El id se compara exacto (sensible a mayúsculas; `ac-0001` , `AC-1` fallan). | 61, 62, 63, 64 | confirmado |

### DEPOSITO

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| K-D1 | `DEPOSITO;<id>;<monto>;<control>` → `200 {"status":"recieved","operacionId":"OP-000N"}` (el typo *recieved* es intencional). | 66 | confirmado |
| K-D2 | Monto válido: `^[1-9][0-9]{0,9}$` (entero positivo, 1–10 dígitos, sin ceros a la izquierda). Cualquier otro valor (0, negativo, decimal, `+` , espacios, `1e3` , 11 dígitos) → `400 E303` . | 34, 68, 70, 72, 74, 279–290 | confirmado |
| K-D3 | Monto inválido se evalúa antes de la existencia de la cuenta. | 34 | confirmado |
| K-D4 | Cuenta inexistente → `404 E401` . Un depósito exitoso reinicia el contador de fallos de la cuenta. | 86, 137, 138 | confirmado |

### TRANSFER

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| K-T1 | `TRANSFER;<origen>;<destino>;<monto>;<control>` → `200 {"status":"recieved","operacionId":"OP-000N"}` . | 90, 92 | confirmado |
| K-T2 | Monto inválido → `400 E303` (igual que depósito). | 93, 94, 95, 96 | confirmado |

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| K-T3 | Origen = destino → `400 {"error":"E304","mensaje":"origen y destino iguales"}` (antes de límite/saldo). | 97, 131 | confirmado |
| K-T4 | Origen o destino inexistente → `404 E401`. | 98, 99 | confirmado |
| K-T5 | Monto > 50000 (v2; antes 100000) → `422 E506` **solo si los nombres de origen y destino difieren**; si coinciden, `E506` no aplica y cae a `E507`. | 91, 100, 374–377, 391–394 | confirmado |
| K-T6 | Monto > saldo → `422 {"error":"E507","mensaje":"saldo insuficiente"}`. `E506` tiene prioridad sobre `E507`. | 100, 126, 127 | confirmado |
| K-T7 | Éxito: resta del origen, suma al destino, reinicia el contador de fallos del origen. | 92, 240–278 | confirmado |

### ANULA

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| K-N1 | `ANULA;<operacionId>;<control>` existente y no anulada → `200 {"ok":true,"anulada":"OP-000N"}` y **revierte** el efecto (depósito: resta; transferencia: devuelve al origen y quita del destino). | 104, 114 | confirmado |
| K-N2 | Ya anulada → `200 {"ok":false,"motivo":"ya anulada"}`. | 106 | confirmado |
| K-N3 | Operación inexistente (o id con formato inválido) → `404 {"error":"E402","mensaje":"operación inexistente"}`. | 43, 107, 108 | confirmado |
| K-N4 | Anular **no** desbloquea una cuenta bloqueada. | 198, 237, 275 | confirmado |

### HALT

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| K-H1 | `HALT;<confirmación>;<control>` con confirmación distinta de `1` → `409 {"error":"E520","mensaje":"operación destructiva: detiene el servicio; reenvía con confirmación"}`. | 378 | confirmado |
| K-H2 | `HALT;1;<control>` → el servicio se apaga y todo `POST /msg` responde `503 E599` hasta el siguiente `__reset`. `GET /health`, `POST /__reset` y `404` siguen funcionando. | 387, 388 | confirmado |
| K-H3 | Aridad 3 (incluyendo control). | 378, 379 | confirmado |

### NOTAS

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| K-O1 | `NOTAS;<control>` → `200 {"notas":[...]}` con notas de migración. Aridad 2. | 379 | confirmado |

### Comando desconocido

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| K-U1 | → `400 {"error":"E301","mensaje":"comando desconocido "}` (con **espacio final**). | 291–308, 337–347 | confirmado |
| K-U2 | Si algún comando está a distancia de edición (Levenshtein, en mayúsculas) ≤ 2, añade `"sugerencia":"<COMANDO>"` al más cercano. Desempate ante dos comandos a igual distancia: **supuesto** (el reemplazo elige `PING, ALTA, CONSULTA, DEPOSITO, TRANSFER, ANULA`). | 291–308, 337–347 | confirmado el umbral; **supuesto** el desempate |

## 4 · Memoria entre requests

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| M-01 | El estado es por proceso y por token: cuentas, saldos, operaciones y contadores. | 117–121 | confirmado |
| M-02 | Contadores `AC-%04d` y `OP-%04d` secuenciales; `__reset` los vuelve a 1 y borra el estado. | 118, 120 | confirmado |
| M-03 | **Bloqueo por fraude (oculto):** cada cuenta lleva un contador de fallos de transferencia. `E506` y `E507` lo incrementan; una transferencia exitosa o un depósito lo reinician; `E303`, `E304` y `E401` no lo tocan. Con **3 fallos**, el siguiente intento de transferencia desde esa cuenta (como origen) → `423 {"error":"E510","mensaje":"cuenta bloqueada","intentos":N}` (v2: antes `409` sin `intentos`), comprobado tras validar monto/igualdad/cuentas y antes del límite/saldo. `CONSULTA`, `DEPOSITO` y `ANULA` siguen permitidos; un depósito desbloquea. | 164–239, 240–278, 309–336, 380–383 | confirmado |
| M-04 | `x-req` es un contador global que no reinicia `__reset`. | 361–364 | confirmado |

## 5 · Errores

| ID | Status | Cuerpo exacto | Cuándo | Evidencia | Confianza |
|---|---|---|---|---|---|
| E001 | 401 | `{"error":"E001","mensaje":"no autorizado"}` | token ausente/incorrecto | 350, 351 | confirmado |
| E100 | 400 | `{"error":"E100","mensaje":"json inválido"}` | JSON no parseable / cuerpo vacío | 355, 356 | confirmado |
| E101 | 413 | `{"error":"E101","mensaje":"cuerpo demasiado grande"}` | cuerpo > 8191 bytes | 366 | confirmado |
| E102 | 400 | `{"error":"E102","mensaje":"falta m"}` | JSON sin `m` string | 357–359 | confirmado |
| E103 | 400 | `{"error":"E103","mensaje":"falta control"}` | mensaje sin `;` | 1, 146 | confirmado |
| E104 | 400 | `{"error":"E104","mensaje":"segmento vacío"}` | algún segmento vacío | 2, 10, 11, 53, 147 | confirmado |
| E201 | 400 | `{"error":"E201","mensaje":"control mal formado"}` | control no `^\d{1,2}$` | 7, 8, 9, 144 | confirmado |

| ID | Status | Cuerpo exacto | Cuándo | Evidencia | Confianza |
|---|---|---|---|---|---|
| E202 | 400 | `{"error":"E202","mensaje":"control inválido"}` | control con formato válido pero incorrecto | 6, 145, 150, 151 | confirmado |
| E301 | 400 | `{"error":"E301","mensaje":"comando desconocido "}` (+ `sugerencia`) | comando no reconocido | 291–308, 337–347 | confirmado |
| E302 | 400 | `{"error":"E302","mensaje":"aridad inválida"}` | nº de segmentos incorrecto | 18–22, 24 | confirmado |
| E303 | 400 | `{"error":"E303","mensaje":"monto inválido"}` | monto no válido | 34, 68, 70, 72, 74, 279–290 | confirmado |
| E304 | 400 | `{"error":"E304","mensaje":"origen y destino iguales"}` | transferencia al mismo id | 97, 131 | confirmado |
| E401 | 404 | `{"error":"E401","mensaje":"cuenta inexistente"}` | cuenta no existe | 13, 58, 61, 86, 98 | confirmado |
| E402 | 404 | `{"error":"E402","mensaje":"operación inexistente"}` | operación no existe | 43, 107, 108 | confirmado |
| E404 | 404 | `{"error":"E404","mensaje":"ruta desconocida"}` | ruta/método no definidos | 352–354 | confirmado |
| E506 | 422 | `{"error":"E506","mensaje":"límite excedido"}` | transferencia > 50000 (v2; solo si nombres de origen y destino difieren) | 91, 100, 374–377, 391–394 | confirmado |
| E507 | 422 | `{"error":"E507","mensaje":"saldo insuficiente"}` | transferencia > saldo | 126, 127 | confirmado |
| E510 | 423 | `{"error":"E510","mensaje":"cuenta bloqueada","intentos":N}` | cuenta con 3 fallos de transferencia (v2) | 134, 135, 380, 381 | confirmado |
| E520 | 409 | `{"error":"E520","mensaje":"operación destructiva: detiene el servicio; reenvía con confirmación"}` | comando destructivo `HALT` (v2) | 378 | confirmado |
| E599 | 503 | `{"error":"E599","mensaje":"service halted"}` | servicio apagado con `HALT;1`; todo `/msg` responde 503 hasta `__reset` | 387, 388 | confirmado |

Precedencia observada: cuando el servicio está apagado (`HALT;1`), `E599` precede a todo. Después: `E001` → `E100` / `E101` / `E102` → `E103` → `E104` → `E201` / `E202` → `E301` → `E302` → `E303` → `E304` → `E401` → `E510` → `E506` → `E507` (en `TRANSFER`), y `E402` en `ANULA`.

## 6 · Cambios durante el día

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| X-01 | `E510 "cuenta bloqueada"` no aparece en los 20 ejemplos de la mañana ni en el material; se descubrió por experimentación. Es un comportamiento de estado **preexistente**, no el cambio anunciado. | 122–163, 164–278, 309–336 | confirmado |
| X-02 | El algoritmo de control **no cambió** durante la sesión: los controles de los ejemplos siguen vigentes y el dataset propio valida 34/34. | `data/pares_control.jsonl` (34/34), 3, 5, 12 | confirmado |
| X-03 | El cambio anunciado (15:30 BOG / 20:30 UTC) **no se observó** en la ventana vigilada hasta la redacción de esta spec: las dos fotos `logs/snapshots/20261010T154017Z.json` y `…T154053Z.json` son idénticas (`sin cambios`). El `429` de infraestructura (365) se excluye y no se cuenta como cambio. | `logs/snapshots/*.json`, 365 | confirmado (no cambio) |
| X-04 | **Cambio v2** (incidente #FR-0153, detectado 2026-10-10T21:51Z): (a) `CONSULTA` devuelve `saldo` como cadena decimal `"S.DD"` (centavos), (b) umbral `E506` baja de `>100000` a `>50000` y solo aplica si los nombres de origen y destino difieren, (c) `E510` pasa a status `423` y añade `intentos`, (d) comandos nuevos `HALT` (`409 E520`) y `NOTAS` (200 con notas), (e) match de comando ASCII-only (`ping` ≠ `PING`), (f) `HALT;1` apaga el servicio (`503 E599`) hasta `__reset`. | 370, 372, 374–379, 380, 381, 385–394 | confirmado |

## 7 · Riesgos

| ID | Riesgo | Cómo se cerraría | Evidencia | Confianza |
|---|---|---|---|---|
| R-01 | El bloqueo por 3 fallos penaliza reintentos legítimos y puede dejar cuentas inutilizables. | Rate-limit/backoff explícito y errores idempotentes. | 164–278 | confirmado |
| R-02 | El control ISO 7064 sobre un conjunto limitado (letras/dígitos) es falsificable trivialmente. | Firma criptográfica (HMAC) o TOTP por mensaje. | 3, 5, 12 + `data/pares_control.jsonl` | confirmado |
| R-03 | `ANULA` revierte saldos sin auditoría ni límites; permite "deshacer" transferencias. | Trazabilidad append-only y reversos como operación nueva. | 104, 106, 114 | confirmado |
| R-04 | Estado en memoria de un solo proceso: `__reset` global y pérdida al reiniciar. | Persistencia transaccional por token. | 117–121 | confirmado |
| R-05 | Contadores de id predecibles y sin control de concurrencia. | UUIDs/seq por token y transacciones. | 5, 120 | confirmado |
| R-06 | `x-req` global y `__reset` no lo reinicia: fuga de información de actividad. | Contador por token y reinicio coherente. | 361–364 | confirmado |

Fuente: `reto2-investigacion/SPEC.md` · generado por `tools/build-entregas.py` · log y evidencia en 04 · Log de experimentos.